Lab Report: Static vs Non-Static Counters in Multithreading

Shawon | ID: IT24051 | Dept. of ICT, MBSTU | Problem SEPM-001

1. Handwritten Code

[Insert the scan/photo of your handwritten code here. Typed reference is on the next page.]

Typed reference of Shawon_Thread.java

```
import java.util.concurrent.atomic.AtomicLong;

public class Shawon_Thread implements Runnable {

    static AtomicLong safeStatic = new AtomicLong(0);  // Experiment A
    static long unsafeStatic = 0;                      // Experiment B
    static boolean threadSafe;
    static long increments;

    long instanceCount = 0;                            // non-static counter

    public void run() {
        for (long i = 0; i < increments; i++) {
            instanceCount++;
            if (threadSafe)
                safeStatic.incrementAndGet();
            else
                unsafeStatic++;
        }
    }

    public static void main(String[] args) throws InterruptedException {
        int n = Integer.parseInt(args[0]);
        increments = Long.parseLong(args[1]);
        threadSafe = Boolean.parseBoolean(args[2]);

        Shawon_Thread[] tasks = new Shawon_Thread[n];
        Thread[] threads = new Thread[n];
        for (int i = 0; i < n; i++) {
            tasks[i] = new Shawon_Thread();
            threads[i] = new Thread(tasks[i]);
            threads[i].start();
        }
        for (int i = 0; i < n; i++)
            threads[i].join();

        long expected = (long) n * increments;
        long staticCount = threadSafe ? safeStatic.get() : unsafeStatic;
        long nonStatic = 0;
        for (Shawon_Thread t : tasks)
            nonStatic += t.instanceCount;

        long diff = Math.abs(staticCount - nonStatic);
        double pct = (nonStatic == 0) ? 0.0 : diff * 100.0 / nonStatic;

        System.out.println("Mode            : " + (threadSafe ? "Thread-safe (AtomicLong)" : "Unsynchronized (long
        System.out.println("Threads (n)     : " + n);
        System.out.println("Increments (m)  : " + increments);
        System.out.println("Expected count  : " + expected);
        System.out.println("Static count    : " + staticCount);
        System.out.println("Non-static total: " + nonStatic);
        System.out.println("Absolute diff   : " + diff);
        System.out.printf("Difference (%%)  : %.4f%%%n", pct);
    }
}
```

2. Test Inputs

Test	Threads (n)	Increments (m)	Expected (n x m)
TC1	1	1,000	1,000
TC2	2	10,000	20,000
TC3	5	10,000	50,000
TC4	10	50,000	500,000
TC5	20	50,000	1,000,000
TC6	50	50,000	2,500,000
TC7	100	50,000	5,000,000

3. Program Output

Experiment A - thread-safe (AtomicLong)

Test	Static (AtomicLong)	Non-static total	Abs. diff	Diff %
TC1	1,000	1,000	0	0.0000%
TC2	20,000	20,000	0	0.0000%
TC3	50,000	50,000	0	0.0000%
TC4	500,000	500,000	0	0.0000%
TC5	1,000,000	1,000,000	0	0.0000%
TC6	2,500,000	2,500,000	0	0.0000%
TC7	5,000,000	5,000,000	0	0.0000%

Experiment B - unsynchronized, all 35 runs

Threads	Run	Static (unsafe)	Non-static total	Abs. diff	Diff %
1	1	1,000	1,000	0	0.0000%
1	2	1,000	1,000	0	0.0000%
1	3	1,000	1,000	0	0.0000%
1	4	1,000	1,000	0	0.0000%
1	5	1,000	1,000	0	0.0000%
2	1	20,000	20,000	0	0.0000%
2	2	20,000	20,000	0	0.0000%
2	3	20,000	20,000	0	0.0000%
2	4	20,000	20,000	0	0.0000%
2	5	20,000	20,000	0	0.0000%
5	1	50,000	50,000	0	0.0000%
5	2	50,000	50,000	0	0.0000%
5	3	50,000	50,000	0	0.0000%
5	4	50,000	50,000	0	0.0000%
5	5	50,000	50,000	0	0.0000%
10	1	366,319	500,000	133,681	26.7362%

Threads	Run	Static (unsafe)	Non-static total	Abs. diff	Diff %
10	2	500,000	500,000	0	0.0000%
10	3	328,835	500,000	171,165	34.2330%
10	4	500,000	500,000	0	0.0000%
10	5	500,000	500,000	0	0.0000%
20	1	1,000,000	1,000,000	0	0.0000%
20	2	1,000,000	1,000,000	0	0.0000%
20	3	1,000,000	1,000,000	0	0.0000%
20	4	1,000,000	1,000,000	0	0.0000%
20	5	1,000,000	1,000,000	0	0.0000%
50	1	2,316,794	2,500,000	183,206	7.3282%
50	2	2,500,000	2,500,000	0	0.0000%
50	3	2,500,000	2,500,000	0	0.0000%
50	4	2,211,944	2,500,000	288,056	11.5222%
50	5	2,239,428	2,500,000	260,572	10.4229%
100	1	5,000,000	5,000,000	0	0.0000%
100	2	5,000,000	5,000,000	0	0.0000%
100	3	5,000,000	5,000,000	0	0.0000%
100	4	5,000,000	5,000,000	0	0.0000%
100	5	5,000,000	5,000,000	0	0.0000%

4. Result Analysis

Threads	Expected	Static (unsafe) - avg of 5	Non-static total	Avg abs. diff	Avg diff %	Runs with loss
1	1,000	1,000	1,000	0	0.0000%	0/5
2	20,000	20,000	20,000	0	0.0000%	0/5
5	50,000	50,000	50,000	0	0.0000%	0/5
10	500,000	439,031	500,000	60,969	12.1938%	2/5
20	1,000,000	1,000,000	1,000,000	0	0.0000%	0/5
50	2,500,000	2,353,633	2,500,000	146,367	5.8547%	3/5
100	5,000,000	5,000,000	5,000,000	0	0.0000%	0/5

Percentage difference = |static - non-static| / non-static x 100

All outputs above are real, unedited program output from running *run_all.sh* (Java 21). They were produced on a machine with 1 CPU core; with a single core threads rarely overlap, so many unsafe runs showed no loss. On a multi-core computer the losses are usually larger and more frequent.

5. Observation and Conclusion

Static vs non-static describes **ownership**, not thread-safety. With AtomicLong, the static counter matched the non-static total in all 7 test cases (0% difference). With a plain static long, lost updates appeared in some runs (TC4: up to 34.23%, TC6: up to 11.52%) while other runs of the same test lost nothing. The difference is **not monotonic** in the thread count: it depends on thread scheduling, number of CPU cores and the JVM's runtime optimisation. The non-static total was always exactly n x m because each thread owns its counter.

6. Analysis Questions

1. What is the difference between a static and a non-static variable in Java?
A static variable belongs to the class: only one copy exists, shared by every object. A non-static (instance) variable belongs to one object: every object gets its own copy.

2. Why do all threads share the same static counter?
The static field is stored once per class, not per object. Every thread runs the same class code, so all of them read and write the same memory location.

3. Why does each thread have its own non-static counter?
Each thread is given its own Shawon_Thread object (created with *new*), and each object has its own *instanceCount*. No other thread ever touches it, so there is no race.

4. Why is join() required before calculating the final counts?
start() returns immediately and the thread runs in the background. Without join(), main could read the counters while threads are still running and get partial values. join() makes main wait until every thread finishes.

5. Why can the unsynchronized static count be lower than expected?
unsafeStatic++ is not one step; it is read, add 1, write back. If two threads read the same old value, both write old+1, and one increment is lost (lost update). Many such overlaps make the final value lower than n x m.

6. Does increasing the number of threads always increase the percentage difference?
No. In my runs the loss appeared for 10 threads (avg 12.19%) and 50 threads (avg 5.85%), but not for 20 or 100 threads. More threads only give more *chance* for overlap; the real loss depends on scheduling, CPU cores and JIT optimisation, so it is not monotonic.

7. Why might two runs with the same number of threads produce different results?
The OS scheduler decides when each thread runs and when it is paused. The interleaving of the read-modify-write steps differs in every run, so the number of lost updates differs (e.g. TC4: 0%, 26.74%, 34.23%).

8. What changes when AtomicLong is used instead of a regular long?
AtomicLong.incrementAndGet() performs the read-modify-write as one indivisible (atomic) operation using hardware compare-and-set, so no update is lost. Static count always equals non-static total and the difference is 0%.

9. How can all threads share one instance counter as well as the static counter?
Create a single object and pass the same object to every Thread: *Shawon_Thread shared = new Shawon_Thread(); new Thread(shared)* for all n threads. Then the instance counter is shared too, so it also needs synchronization (a synchronized method or AtomicLong).

Appendix: raw program output

```
=== TC1 : threads=1, increments=1000 ===
--- Thread-safe (true) ---
Mode            : Thread-safe (AtomicLong)
Threads (n)     : 1
Increments (m)  : 1000
Expected count  : 1000
Static count    : 1000
Non-static total: 1000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 1 ---
Mode            : Unsynchronized (long)
Threads (n)     : 1
Increments (m)  : 1000
Expected count  : 1000
Static count    : 1000
Non-static total: 1000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 2 ---
Mode            : Unsynchronized (long)
Threads (n)     : 1
Increments (m)  : 1000
Expected count  : 1000
Static count    : 1000
Non-static total: 1000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 3 ---
Mode            : Unsynchronized (long)
Threads (n)     : 1
Increments (m)  : 1000
Expected count  : 1000
Static count    : 1000
Non-static total: 1000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 4 ---
Mode            : Unsynchronized (long)
Threads (n)     : 1
Increments (m)  : 1000
Expected count  : 1000
Static count    : 1000
Non-static total: 1000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 5 ---
Mode            : Unsynchronized (long)
Threads (n)     : 1
Increments (m)  : 1000
Expected count  : 1000
Static count    : 1000
Non-static total: 1000
Absolute diff   : 0
Difference (%)  : 0.0000%

=== TC2 : threads=2, increments=10000 ===
--- Thread-safe (true) ---
Mode            : Thread-safe (AtomicLong)
Threads (n)     : 2
Increments (m)  : 10000
Expected count  : 20000
Static count    : 20000
Non-static total: 20000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 1 ---
Mode            : Unsynchronized (long)
Threads (n)     : 2
Increments (m)  : 10000
Expected count  : 20000
Static count    : 20000
Non-static total: 20000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 2 ---
Mode            : Unsynchronized (long)
Threads (n)     : 2
Increments (m)  : 10000
Expected count  : 20000
Static count    : 20000
Non-static total: 20000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 3 ---
Mode            : Unsynchronized (long)
Threads (n)     : 2
Increments (m)  : 10000
Expected count  : 20000
Static count    : 20000
```

```
Non-static total: 20000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 4 ---
Mode            : Unsynchronized (long)
Threads (n)     : 2
Increments (m)  : 10000
Expected count  : 20000
Static count    : 20000
Non-static total: 20000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 5 ---
Mode            : Unsynchronized (long)
Threads (n)     : 2
Increments (m)  : 10000
Expected count  : 20000
Static count    : 20000
Non-static total: 20000
Absolute diff   : 0
Difference (%)  : 0.0000%

=== TC3 : threads=5, increments=10000 ===
--- Thread-safe (true) ---
Mode            : Thread-safe (AtomicLong)
Threads (n)     : 5
Increments (m)  : 10000
Expected count  : 50000
Static count    : 50000
Non-static total: 50000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 1 ---
Mode            : Unsynchronized (long)
Threads (n)     : 5
Increments (m)  : 10000
Expected count  : 50000
Static count    : 50000
Non-static total: 50000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 2 ---
Mode            : Unsynchronized (long)
Threads (n)     : 5
Increments (m)  : 10000
Expected count  : 50000
Static count    : 50000
Non-static total: 50000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 3 ---
Mode            : Unsynchronized (long)
Threads (n)     : 5
Increments (m)  : 10000
Expected count  : 50000
Static count    : 50000
Non-static total: 50000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 4 ---
Mode            : Unsynchronized (long)
Threads (n)     : 5
Increments (m)  : 10000
Expected count  : 50000
Static count    : 50000
Non-static total: 50000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 5 ---
Mode            : Unsynchronized (long)
Threads (n)     : 5
Increments (m)  : 10000
Expected count  : 50000
Static count    : 50000
Non-static total: 50000
Absolute diff   : 0
Difference (%)  : 0.0000%

=== TC4 : threads=10, increments=50000 ===
--- Thread-safe (true) ---
Mode            : Thread-safe (AtomicLong)
Threads (n)     : 10
Increments (m)  : 50000
Expected count  : 500000
Static count    : 500000
Non-static total: 500000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 1 ---
Mode            : Unsynchronized (long)
Threads (n)     : 10
Increments (m)  : 50000
Expected count  : 500000
```

```
Static count    : 366319
Non-static total: 500000
Absolute diff   : 133681
Difference (%)  : 26.7362%
--- Unsynchronized (false), run 2 ---
Mode            : Unsynchronized (long)
Threads (n)     : 10
Increments (m)  : 50000
Expected count  : 500000
Static count    : 500000
Non-static total: 500000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 3 ---
Mode            : Unsynchronized (long)
Threads (n)     : 10
Increments (m)  : 50000
Expected count  : 500000
Static count    : 328835
Non-static total: 500000
Absolute diff   : 171165
Difference (%)  : 34.2330%
--- Unsynchronized (false), run 4 ---
Mode            : Unsynchronized (long)
Threads (n)     : 10
Increments (m)  : 50000
Expected count  : 500000
Static count    : 500000
Non-static total: 500000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 5 ---
Mode            : Unsynchronized (long)
Threads (n)     : 10
Increments (m)  : 50000
Expected count  : 500000
Static count    : 500000
Non-static total: 500000
Absolute diff   : 0
Difference (%)  : 0.0000%

=== TC5 : threads=20, increments=50000 ===
--- Thread-safe (true) ---
Mode            : Thread-safe (AtomicLong)
Threads (n)     : 20
Increments (m)  : 50000
Expected count  : 1000000
Static count    : 1000000
Non-static total: 1000000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 1 ---
Mode            : Unsynchronized (long)
Threads (n)     : 20
Increments (m)  : 50000
Expected count  : 1000000
Static count    : 1000000
Non-static total: 1000000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 2 ---
Mode            : Unsynchronized (long)
Threads (n)     : 20
Increments (m)  : 50000
Expected count  : 1000000
Static count    : 1000000
Non-static total: 1000000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 3 ---
Mode            : Unsynchronized (long)
Threads (n)     : 20
Increments (m)  : 50000
Expected count  : 1000000
Static count    : 1000000
Non-static total: 1000000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 4 ---
Mode            : Unsynchronized (long)
Threads (n)     : 20
Increments (m)  : 50000
Expected count  : 1000000
Static count    : 1000000
Non-static total: 1000000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 5 ---
Mode            : Unsynchronized (long)
Threads (n)     : 20
Increments (m)  : 50000
Expected count  : 1000000
Static count    : 1000000
```

```
Non-static total: 1000000
Absolute diff   : 0
Difference (%)  : 0.0000%

=== TC6 : threads=50, increments=50000 ===
--- Thread-safe (true) ---
Mode            : Thread-safe (AtomicLong)
Threads (n)     : 50
Increments (m)  : 50000
Expected count  : 2500000
Static count    : 2500000
Non-static total: 2500000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 1 ---
Mode            : Unsynchronized (long)
Threads (n)     : 50
Increments (m)  : 50000
Expected count  : 2500000
Static count    : 2316794
Non-static total: 2500000
Absolute diff   : 183206
Difference (%)  : 7.3282%
--- Unsynchronized (false), run 2 ---
Mode            : Unsynchronized (long)
Threads (n)     : 50
Increments (m)  : 50000
Expected count  : 2500000
Static count    : 2500000
Non-static total: 2500000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 3 ---
Mode            : Unsynchronized (long)
Threads (n)     : 50
Increments (m)  : 50000
Expected count  : 2500000
Static count    : 2500000
Non-static total: 2500000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 4 ---
Mode            : Unsynchronized (long)
Threads (n)     : 50
Increments (m)  : 50000
Expected count  : 2500000
Static count    : 2211944
Non-static total: 2500000
Absolute diff   : 288056
Difference (%)  : 11.5222%
--- Unsynchronized (false), run 5 ---
Mode            : Unsynchronized (long)
Threads (n)     : 50
Increments (m)  : 50000
Expected count  : 2500000
Static count    : 2239428
Non-static total: 2500000
Absolute diff   : 260572
Difference (%)  : 10.4229%

=== TC7 : threads=100, increments=50000 ===
--- Thread-safe (true) ---
Mode            : Thread-safe (AtomicLong)
Threads (n)     : 100
Increments (m)  : 50000
Expected count  : 5000000
Static count    : 5000000
Non-static total: 5000000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 1 ---
Mode            : Unsynchronized (long)
Threads (n)     : 100
Increments (m)  : 50000
Expected count  : 5000000
Static count    : 5000000
Non-static total: 5000000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 2 ---
Mode            : Unsynchronized (long)
Threads (n)     : 100
Increments (m)  : 50000
Expected count  : 5000000
Static count    : 5000000
Non-static total: 5000000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 3 ---
Mode            : Unsynchronized (long)
Threads (n)     : 100
Increments (m)  : 50000
Expected count  : 5000000
```

```
Static count    : 5000000
Non-static total: 5000000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 4 ---
Mode            : Unsynchronized (long)
Threads (n)     : 100
Increments (m)  : 50000
Expected count  : 5000000
Static count    : 5000000
Non-static total: 5000000
Absolute diff   : 0
Difference (%)  : 0.0000%
--- Unsynchronized (false), run 5 ---
Mode            : Unsynchronized (long)
Threads (n)     : 100
Increments (m)  : 50000
Expected count  : 5000000
Static count    : 5000000
Non-static total: 5000000
Absolute diff   : 0
Difference (%)  : 0.0000%
```